DUNGEON SERPENT

System Design Document

Ray Morrow

Washington State University
CptS 322 — Software Engineering Principles I

Version:	1.0
Semester:	Spring 2026
Implementation:	C# / .NET 8 / MonoGame 3.8.1
Document Type:	System Design Document (SDD)

Prepared for Assignment 3 — System Design Document
WSU CptS 322, Spring 2026

Contents

1 Introduction

1.1 Purpose

This document describes the design of **Dungeon Serpent**, a 2D single-player snake game built in C# using the MonoGame 3.8.1 framework. The Software Requirements Specification (SRS) covered *what* the game needs to do; this System Design Document (SDD) covers *how* it is actually put together. That includes the finite state machine that drives screen transitions, the flow of data from file to gameplay and back, the architectural decisions that shape the codebase, and the class, sequence, and flowchart diagrams that make those decisions concrete.

It is meant to be useful during implementation — not as a rigid contract, but as a reference that answers "how does X work again?" without digging through code.

1.2 Reference Documents

- MonoGame Foundation. (2024). *MonoGame Documentation, v3.8.1.* https://docs.monogame.net/
- Microsoft. (2024). *.NET 8 Documentation.* https://learn.microsoft.com/en-us/dotnet/
- Microsoft. (2024). *C# Language Reference.* https://learn.microsoft.com/en-us/dotnet/csharp/
- Nystrom, R. (2014). *Game Programming Patterns.* https://gameprogrammingpatterns.com/
- Fowler, M. (2003). *Patterns of Enterprise Application Architecture.* Addison-Wesley.
- WSU CptS 322 Assignment 3 Rubric. Provided by course instructor, Spring 2026.
- WSU CptS 322 Dungeon Serpent SRS. Authored by Ray Morrow, Spring 2026.

1.3 Abbreviations and Acronyms

SDD
: System Design Document

SRS
: Software Requirements Specification

FSA
: Finite State Automaton

DFD
: Data Flow Diagram

UML
: Unified Modeling Language

MVC
: Model-View-Controller

HUD
: Heads-Up Display (the in-game info bar)

FPS
: Frames Per Second

I/O
Input / Output

DGL
DesktopGL (MonoGame's cross-platform desktop target)

JSON
JavaScript Object Notation

API
Application Programming Interface

2 Finite State Automata

Dungeon Serpent has five discrete states. Figure 1 shows the FSA; the descriptions below explain what each state does and when it transitions.

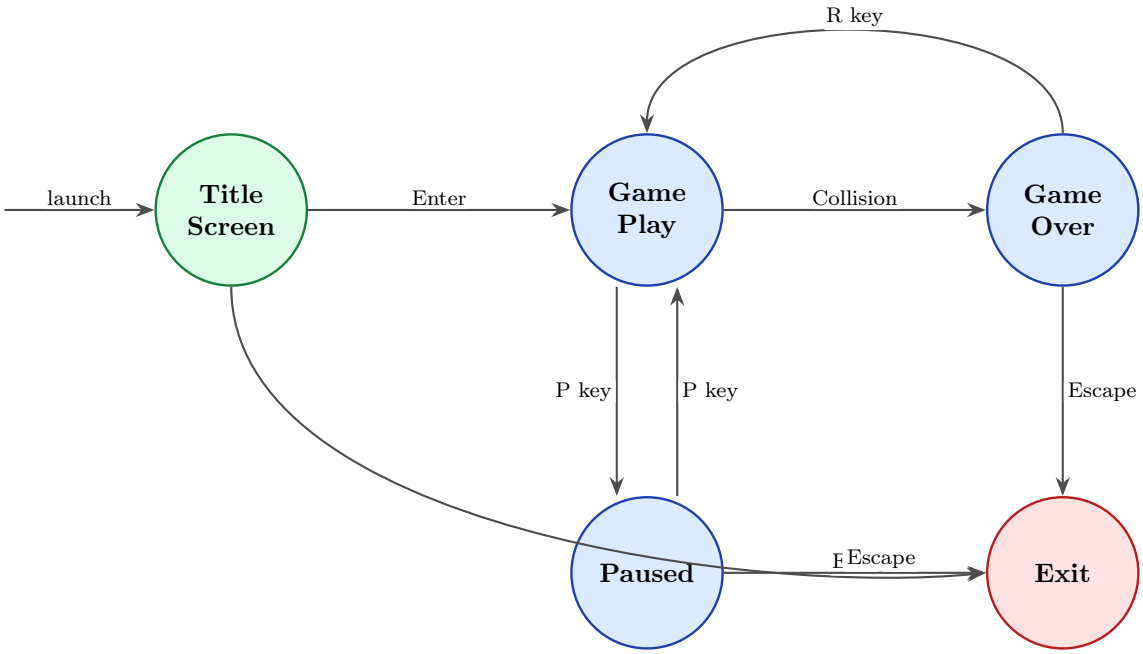

Figure 1: Dungeon Serpent Finite State Automaton.

State descriptions:

- **Title Screen** — The game starts here every time it launches. The dungeon floor and torch animations run in the background while the title pulses on screen. Pressing `Enter` moves to Game Play; pressing `Escape` jumps straight to Exit. Nothing about the actual game state is set until the player commits.
- **Game Play** — The active loop. The serpent moves on a fixed step timer; all collision detection, gem spawning, scoring, and level-up logic runs here. It is the only state where the player's keyboard input directly affects what happens on the grid. Pressing `P` suspends it into Paused; a wall or self-collision transitions to Game Over.
- **Paused** — All game logic stops. The dungeon frame is still visible under a semi-transparent overlay showing pause instructions. No timer advances, no particles update, nothing changes. Pressing `P` again drops back into Game Play exactly where it left off. `Escape` goes to Exit.
- **Game Over** — Entered the moment a fatal collision is detected. The final score is displayed and the player has two choices: `R` calls `ResetGame()` and returns to Game Play with a fresh session, or `Escape` exits. Directional keys are ignored here.
- **Exit** — The terminal state. `Game.Exit()` fires, MonoGame disposes all managed resources (`SpriteBatch`, `Texture2D`, `SpriteFont`, `GraphicsDevice`), and the process ends. There is no confirmation dialog.

3 Data Flow

Figure 2 traces data through the system from the moment the application starts to the moment it shuts down. The diagram uses two visual conventions: **blue sharp-cornered rectangles** for processing steps, and **green rounded rectangles** for data stores. An external entity (the player) feeds keyboard input into the system.

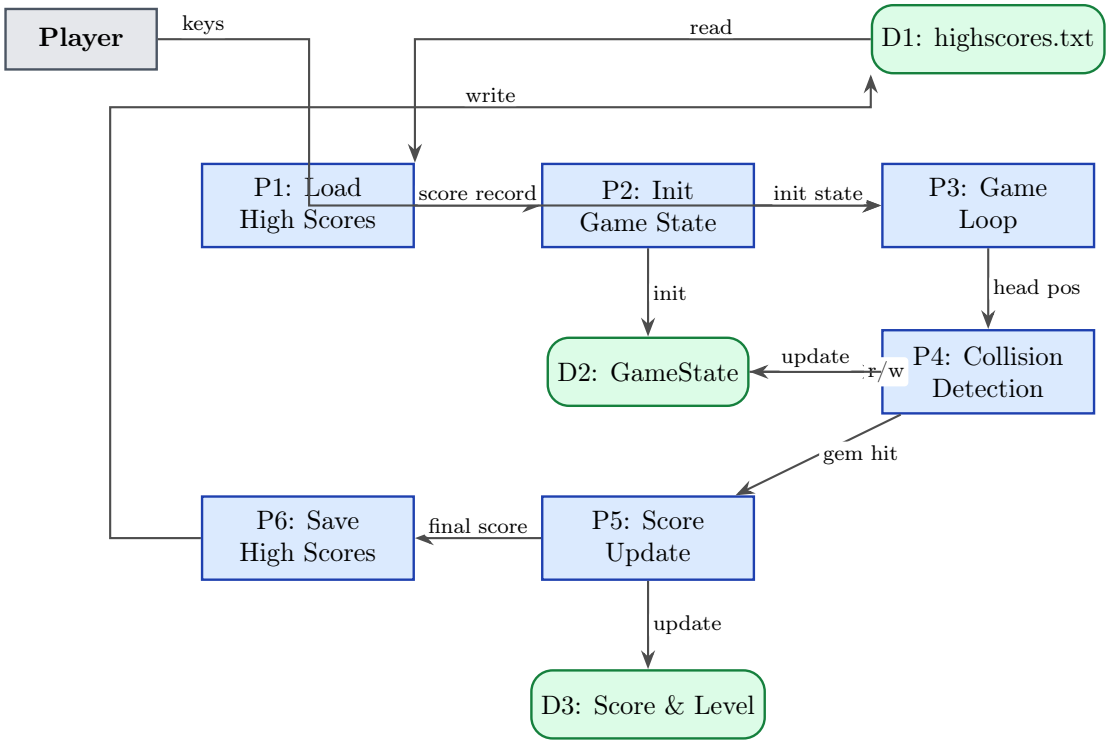

Figure 2: Dungeon Serpent Data Flow Diagram.

Processing Step Descriptions

Step	Inputs	Actions	Outputs
P1: Load High Scores	`highscores.txt` (D1); default values if the file is missing	Open the plaintext file, parse each line, validate the values. If the file does not exist, create it on first write. Fall back to a zero-score default on any read error.	A score record passed to P2 for initialization
P2: Init Game State	Score record from P1; hardcoded constants (grid dimensions, starting speed, gem table)	Build the serpent `LinkedList` at the grid center. Set direction to right, score to 0, level to 1. Spawn the first gem at a random valid position.	A fully initialized `GameState` written to D2

Step	Inputs	Actions	Outputs
P3: Game Loop	Keyboard input from the player; current `GameState` from D2; delta time	Read direction keys and update `_nextDir`. Advance the step timer. When the timer fires, call `StepSnake()`. Update particle and message lists. Render the frame.	Updated head position; a rendered frame on screen
P4: Collision Detection	New head position; wall boundary constants; current serpent body list; gem position	Test the new head against each wall boundary. Test it against every occupied body segment. Test it against the gem position. Set the appropriate flag.	`GameOver = true` or `GemEaten = true`; updated D2 (serpent grew, or session ended)
P5: Score Update	`GemEaten` event; gem base point value; current level; running score in D3	Multiply gem base points by the current level. Add the result to the running score. Check whether the 5-gem level-up threshold has been crossed. If so, increment level and reduce `_stepInterval`.	Updated score and level in D3; a level-up event if the threshold was met
P6: Save High Scores	Final score from P5; existing top-score list from D3	Insert the new score into the list, sort descending, trim to the top 5. Write the result back to `highscores.txt`.	Updated `highscores.txt` (D1) for the next session

4 Architecture

Dungeon Serpent is built on three patterns that fit naturally together: a **Game Loop** for the core execution cycle, **Model-View-Controller (MVC)** for separating data from rendering from logic, and a **State Pattern** for managing which screen is active. Figure 3 shows how the major components connect.

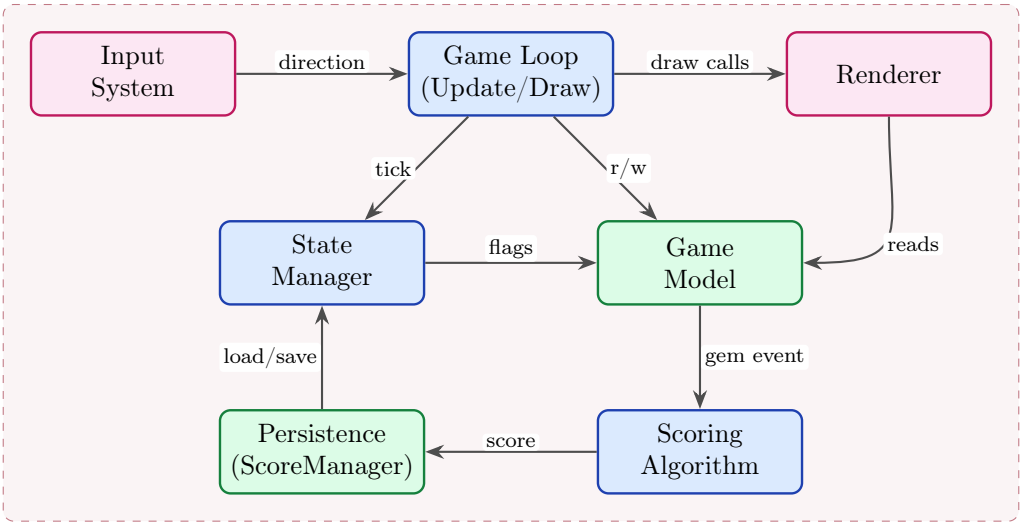

Figure 3: Dungeon Serpent Component Architecture Diagram.

Pattern 1: Game Loop

MonoGame calls `Update()` each frame. The game loop sits inside that method: it reads keyboard state, advances a step timer, and when the timer fires, runs a single game-logic step (move the serpent, check collisions, update score). `Draw()` runs separately and only reads state — it never mutates anything.

The step timer is what keeps the game playable at any render FPS. At level 1, a step fires every 0.125 seconds (8 steps per second). By level 6 that's down to around 0.05 seconds. The render loop always runs at 60 FPS — it just has less interesting work to do between logic steps.

The reason to use this pattern is simple: MonoGame is built around it. Every `Game` subclass gets `Update()` and `Draw()` for free. The trade-off is that logic and rendering share a thread. A slow `Update()` will drop frames. For Dungeon Serpent, where each update does at most a `LinkedList` traversal and a handful of arithmetic operations, this is not a real concern.

Pattern 2: Model-View-Controller (MVC)

The code is organized into three responsibilities that do not bleed into each other.

The **Model** is the game's data: the serpent `LinkedList<Point2>`, the gem position and type, the current score, the current level, the particle list, and the floating message list. Nothing in the Model knows how to draw itself or how to respond to input.

The **View** is the collection of draw helpers: `DrawFloor()`, `DrawSnake()`, `DrawGem()`, `DrawTorches()`, `DrawHUD()`, `DrawTitle()`, and `DrawOverlay()`. These methods only read state. They never touch `_snake` to add a segment or change `_score`.

The **Controller** is `Update()` and `StepSnake()`. These are the only places where state changes. Input comes in, the timer advances, and the Model gets updated.

MVC was the right call here because it makes the code easier to reason about. If a rendering bug appears, the cause is in a `Draw*()` method. If a gameplay bug appears, the cause is in `Update()` or `StepSnake()`. The patterns do not overlap. The main trade-off is slightly more structure than a purely procedural approach, but at this project's size that cost is minimal.

Pattern 3: State Pattern

Five game states, each with distinct behavior, are managed by three boolean flags: `_titleScreen`, `_paused`, and `_gameOver`. At any given frame, the combination of these flags determines which branch of `Update()` and `Draw()` runs. Only one active state is possible at a time.

Transitions are edge-triggered: the input system checks whether a key is newly pressed this frame (not just held), so a single press cannot cascade through multiple states.

The main advantage over a chain of `if/else` blocks is isolation. Adding a new screen — say, a settings menu — means adding a new flag and handling it in its own isolated block, without touching the Game Play or Pause logic. The down side compared to a full `IGameState` interface is less formal polymorphism, but for five states that is not a meaningful limitation.

5 Structural Diagrams

5.1 UML Class Diagram

Figure 4 shows the primary class structure of Dungeon Serpent. The diagram covers all five areas required by the assignment template: screens/controller, view helpers, algorithm and I/O, and game data.

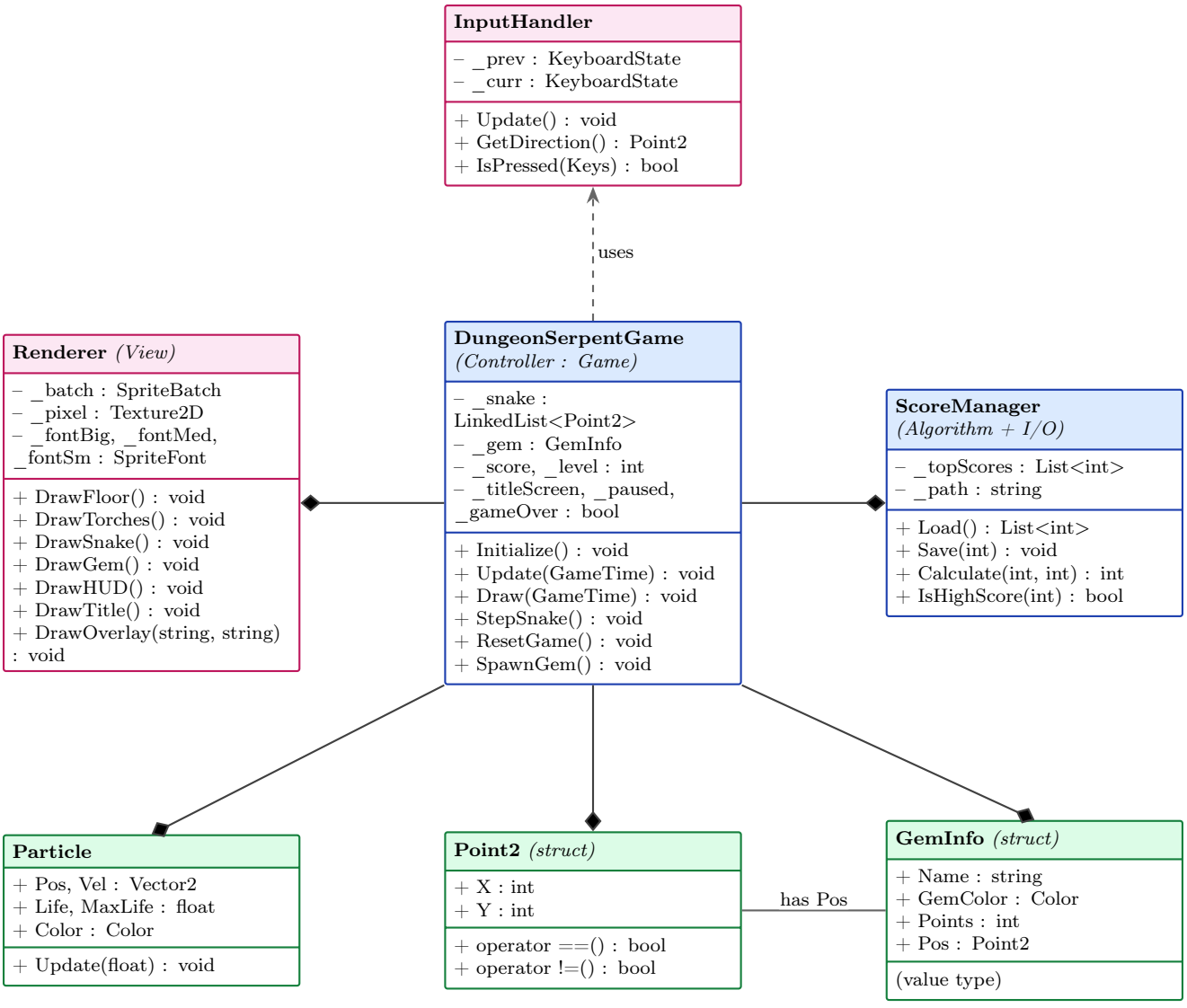

Figure 4: Dungeon Serpent UML Class Diagram.

DungeonSerpentGame extends MonoGame's `Game` class and acts as the central controller. It owns the game loop, holds all shared state, and calls into the other classes. Its primary fields are the serpent `LinkedList<Point2>`, the current `GemInfo`, score, level, and the three state flags. Key methods include `Initialize()`, `Update(GameTime)`, `Draw(GameTime)`, `StepSnake()`, `ResetGame()`, and `SpawnGem()`.

Renderer groups all view-layer drawing logic. It holds `SpriteBatch`, the 1x1 white `Texture2D` used for all primitive drawing, and the three `SpriteFont` instances. Its relationship to the con-

troller is a *dependency* (dashed arrow): the controller calls its methods, but `Renderer` does not hold a reference back. Primary methods: `DrawFloor()`, `DrawTorches()`, `DrawSnake()`, `DrawGem()`, `DrawHUD()`, `DrawTitle()`, `DrawOverlay(string, string)`.

ScoreManager handles the scoring algorithm and all file I/O. It is composed by the controller (filled diamond). Primary methods: `Load()` returns the saved top-score list, `Save(int)` inserts a new score and persists the list, `Calculate(int base, int level)` returns `base * level`, and `IsHighScore(int)` compares against the current top.

GemInfo is a struct holding the data for one gem: `Name`, `GemColor`, `Points`, and `Pos` (`Point2`). The controller uses composition to hold exactly one active gem at a time.

Point2 is a value type with `X` and `Y` integer fields and overloaded `==` and `!=` operators. It is used for every grid coordinate in the system — serpent segments, gem position, and directional vectors.

Particle holds the state for a single particle effect: position, velocity, life, max life, and color. The controller owns a `List<Particle>` (composition).

InputHandler wraps MonoGame's `KeyboardState` to provide edge-triggered key detection. It stores the previous frame's state and exposes `GetDirection()` and `IsPressed(Keys)`.

Associations used in the diagram: *composition* (filled diamond) for objects the controller creates and owns, *dependency* (dashed arrow) for objects the controller uses but does not own, and a plain *association* line for `GemInfo` referencing `Point2` for its position field.

6 Behavioural Diagrams

6.1 UML Sequence Diagram: Game Play Start and End

Figure 5 models two sections of the session lifecycle: **initialization** (from launch to the first game step) and the **game loop** (the repeating cycle of input, movement, collision, and rendering that runs until a collision ends the session). Three actors are involved: the Player, the Game System, and the File System.

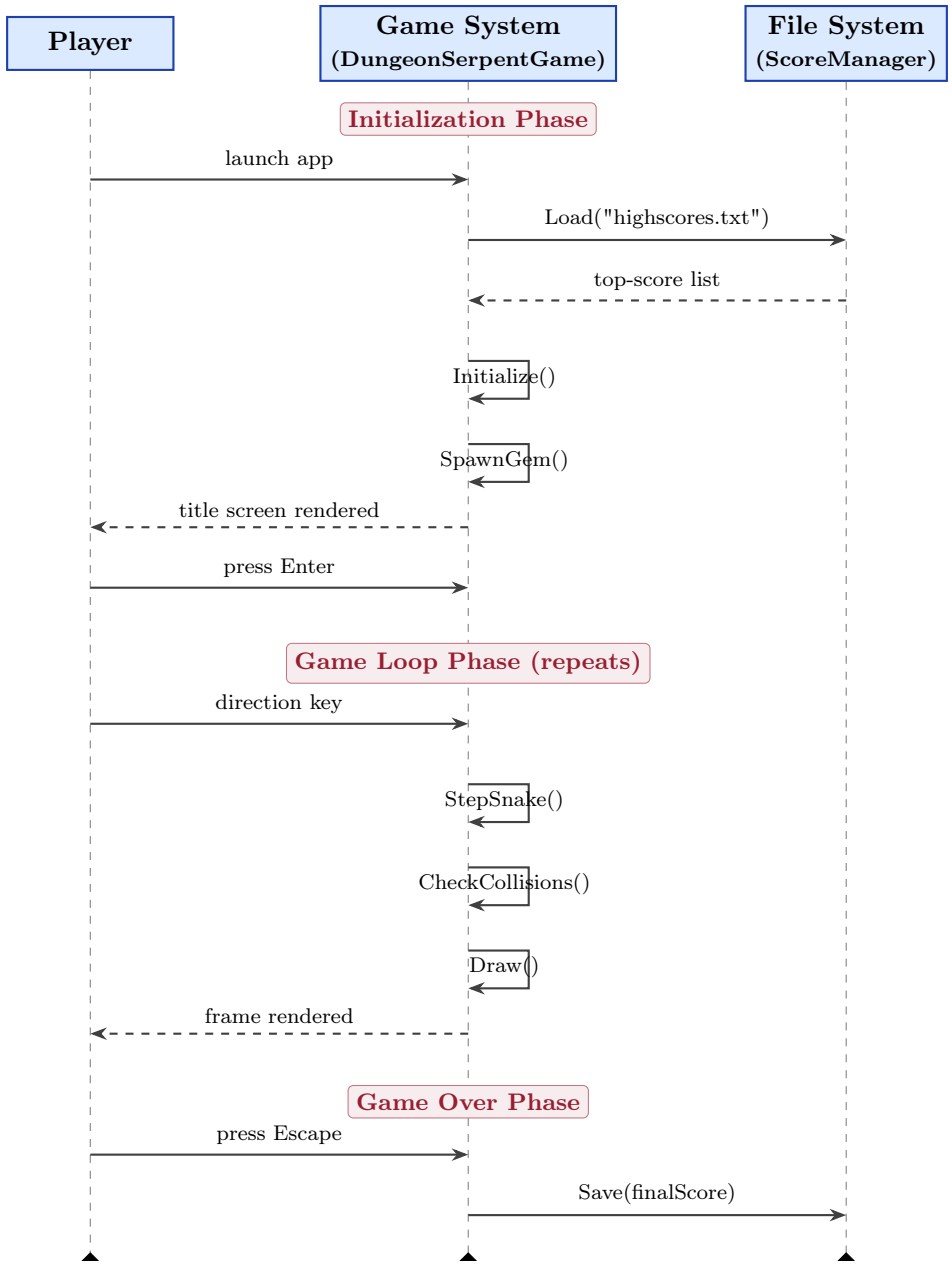

Figure 5: UML Sequence Diagram — Game Play Start and End.

On launch, the Game System immediately calls `Load("highscores.txt")` on the File System and waits for the score list to come back. With that in hand, it calls `Initialize()`: the serpent is

built, direction is set to right, score and level are reset to their starting values, and the first gem is spawned at a random valid position. The title screen then renders until the player presses `Enter`.

Once in Game Play, the sequence repeats every step interval. The Player sends a direction key (or does nothing, in which case the last direction persists). The Game System reads that input and updates `_nextDir`. It then calls `StepSnake()`, which moves the head, runs wall and self collision checks, and runs the gem collision check. If a gem was hit, the serpent grows, a new gem spawns, and the Score Update logic fires. At the end of the step, `Draw()` renders everything to the screen.

When a collision ends the session, the Game System sets `_gameOver = true` and shows the overlay. The player presses `Escape` or `R`. If they press `Escape`, the Game System calls `Save(finalScore)` on the File System before calling `Exit()`. If they press `R`, `ResetGame()` runs and the loop starts again — no file write occurs mid-session.

The sequence diagram object lifelines match the class diagram directly: the Game System lifeline corresponds to `DungeonSerpentGame`, the File System lifeline corresponds to `ScoreManager`, and input messages originate from `InputHandler` on behalf of the Player.

6.2 Game Play Flow Chart

Figure 6 shows the complete logic flow for the active Game Play state, from the moment the session is initialized to the moment it ends (Game Over). The chart uses the standard UML flowchart notation: a solid black dot for the start of flow, a circled black dot for the end, diamonds for decision points (with Yes/No labels on each branch), and rectangles for process steps.

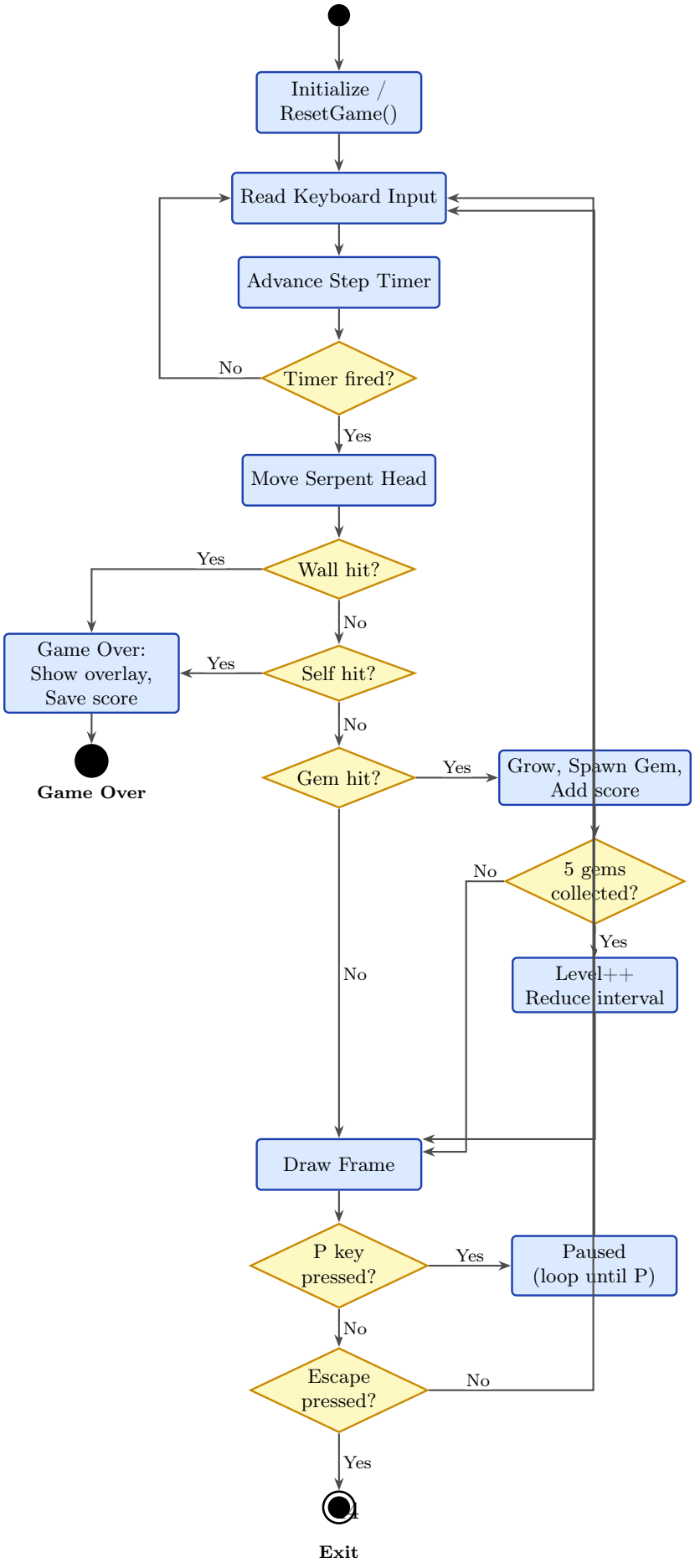

Figure 6: Dungeon Serpent Game Play Flow Chart.

The game starts by running `ResetGame()`, which sets up the serpent, score, gem, and speed. After that, every iteration of the main loop follows the same path. First, keyboard input is read and the direction variable is updated if a valid (non-reversing) key was pressed. The step timer advances by the frame's delta time. If the timer has not yet reached the step interval, the logic skips straight to rendering.

When the step timer does fire, the serpent head moves one tile in the current direction. Three collision checks run in order: wall first, self second, gem third. Wall and self collisions both set `_gameOver = true` and route to the Game Over block, which shows the overlay and saves the final score. A gem collision grows the serpent by one segment, spawns a new gem at a random valid position, and calls the scoring function (`base * level`). A sub-decision then checks whether the 5-gem level-up threshold has been crossed; if so, level increments and the step interval decreases.

After the collision block, the frame is drawn: floor tiles, torches, gem with its bob animation, serpent with scale and eye detail, particles, floating messages, and the HUD. Two more checks close the loop: if `P` was pressed, logic halts and the Paused overlay shows (the loop resumes from the same point when `P` is pressed again); if `Escape` was pressed, the application exits. If neither fires, the loop returns to the input read step.

The chart has no dead ends and no contradictions. Every branch eventually reaches either the Game Over end state, the Exit end state, or loops back to the input read step.